Estela: shipment visibility

Federico Javier Martino. Write-up for the Shipment Tracking App technical test. The prototype is in the repository; its README opens with one command and an eight-step demo.

---

1. The cut

Thesis. The manufacturer's problem is not a missing dashboard; it is that nobody can trust a status without knowing who said it. So the product is one timeline per shipment in which every status and every date carries its provenance, and two views built on it: operations work a short queue of cases that need a person, and customers get a verdict and hear about a delay from us first.

Deep, and why.

1. *Consolidation with provenance.* Four operators, four formats (a haulier's CSV in Spanish, a German JSON webhook, a DCSA-style ocean API, a forwarder's daily report and free-text emails) are parsed by one adapter each and folded into a single timeline; the raw payload sits one click under every event. "Never invent; distinguish confirmed from estimated" is the brief's hardest claim, so types and tests are concentrated here.
2. *Management by exception.* The queue is ranked by the deadline that still changes the outcome (an export cut-off, tonight's linehaul, the end of free time at the terminal), not by severity. Each case prints its evidence, its clock and the next step.
3. *One loop across both roles.* A vessel is delayed; Estela re-estimates the door date before the forwarder does; the case enters the queue with a drafted notice; a named person edits and approves; the customer's verdict and date change. When the operator later confirms the same date, the label changes from "Estela estimate" to "Operator estimate" and no second notice is proposed.

Light, one interaction each: natural-language search, the daily briefing, documents, live updates (four scripted operator events sent from a demo bar) and the customer portal, which reuses the same components read-only.

Out, on purpose. A map (it answers neither role's question; a route strip gives position in 40 px). An open-ended assistant (unbounded, impossible to evaluate, and it invites answers the data cannot support). Login and user admin (a persona switcher proves the perimeter faster). KPI dashboards (they report the past; the thesis is acting today). Assignment, comments, snooze, notification settings, mobile layouts, dark mode, a translated UI and a second ocean lane with transhipment: each costs polish elsewhere without proving the thesis.

A small, checkable world. 23 shipments from 3 sites to 5 accounts; one ocean lane (Valencia to Veracruz) plus domestic and EU road; a demo clock frozen on a working day (Wed 7 Oct 2026, 16:00 in Zaragoza), because a load-time clock produces Saturday deliveries and 3 a.m. customs releases. Both queries in the brief work verbatim: order 12345 is the hero shipment.

---

2. Where and how AI adds value

AI in Estela is an attention and language layer over a ledger it cannot write to. Rules decide what is a fact, what is an exception and which step to propose, because those must be auditable; models estimate, read and write. Each capability sits behind a port with a deterministic mock, so the prototype shows where the model lives and what surrounds it.

Capability	What the user gets	Prototype	Production	Guard
Estimate (ETA and risk)	A door date with a window and the chain of steps behind it; "At risk" before any operator says so; or *withheld*, with the reason	Rule-based propagation over the plan, labelled as such	Per-lane learned model (quantile regression on lane, operator, season, current slip), in shadow until it beats the operator's ETA; the rules remain the fallback	No percentages without history; abstains on stale data
Reading free text	A customs hold read from a broker's Spanish email	Pattern rules	Small language model constrained to the closed fact schema; accepted readings become mapping rules	A person confirms before it becomes a fact or reaches the customer
Briefing	"6 need you: 1 delayed, 2 held, 2 at risk, 1 stale", plus one sentence on where acting today changes the outcome	Counts are queries; the sentence is a template	Same counts; a small model writes the sentence	Counts are never generated

Capability	What the user gets	Prototype	Production	Guard
Drafts	A customer notice or a message to an operator, ready to edit	Templates filled from a fact sheet	Larger language model whose only context is the fact sheet for that audience	Named approver; the published date comes from a snapshot, not from the prose; a lint flags dates in the text that are not in the facts
Search and ask	"shipments to France this week running late" becomes editable filter chips; "order 12345?" gets a status line; a question about duty gets an honest refusal	Grammar parser	Small model emitting the typed filter (closed enums; periods resolved by code)	The model can only produce a filter, never a row; scope is applied after it

How it is presented. One visual grammar in both roles: solid for what an operator confirmed, dashed for what an operator estimates, dotted violet for what Estela's model says, and an `AI` tag on generated text. Violet means "a model produced this" and nothing else: computed counts, holds and playbook steps are not labelled AI, so the label keeps meaning "this may be wrong". Two publishing rules follow. *Facts flow, predictions wait*: operator facts reach the customer immediately; a model estimate only inside a notice someone approved. *No green lie*: the customer never reads "On time" while Estela expects a miss; the verdict goes neutral until a person decides.

What I did not give to a model: choosing exceptions and actions (a playbook the operations lead can read and change); a language model in the read path (asking, drafting and the briefing sentence run on a person's action; the prototype recomputes estimates on read and caches them, where production would compute them at ingestion and store them); outbound messages without approval. Auto-publishing is earned per notice class, once its approved-without-edit rate is known.

Taking it to production. Rollout order: (1) the ledger with rule-based normalisation, exceptions and estimates: no model, already consolidated visibility, and the history later models need; (2) the search interpreter, whose errors are visible and cheap; (3) drafts with mandatory approval; (4) model reading of free text behind confirmation; (5) the learned estimator, lane by lane. Evaluation is already seeded: the query cases, the check that every date in a draft exists in its fact sheet and the estimator invariants are written against the ports as data, so pointing the same runner at a real adapter turns the unit suite into the offline evaluation set. Online signals: lead time over the operator's own ETA, drafts approved unedited, readings rejected, chips edited. Language models are called per human action or per new operator phrase, never per event or page view, so latency and correctness are the real budgets, not cost.

---

3. Technical approach

Stack. Next.js 16 (App Router), React 19, strict TypeScript, Tailwind v4; TanStack Query over an async in-browser gateway; Zod at the operator boundary; Vitest and Playwright. It is the stack this would ship on, and it is worth being honest about what it buys in a frontend-only prototype: layouts per role, routing and URL state, error and not-found boundaries. All domain data lives in the browser, so each role layout renders behind a client-only boundary; in production the gateway's reads move to Server Components and its commands to Server Actions without the screens changing. A Vite SPA would have started marginally faster and left that path undrawn. There are no Route Handlers or Server Actions: a fake API is a backend.

Hexagonal, with restraint. `domain` (pure TypeScript) ← `application` (ports and the gateway) ← `adapters` (operators, stores, mock AI) ← `composition` ← `ui`. The dependency rule and a ban on `Date.now()` outside the clock adapters and the composition root are enforced by ESLint, not by convention. Plain data and pure functions; no container, no event bus.

```
operator payloads      CSV, JSON webhook, API, daily report, email
       |               one adapter per operator: Zod schema + mapping table; free text -> AI port
       v
append-only log        raw messages kept verbatim     <---  the user's commands append here too
       |               pure fold
       v
timeline, stage, three dates, exceptions and steps
       |
       v
Estela gateway         perimeter, audience view types        --->  React
```

Decisions I would defend.

- *An append-only log and a pure fold* instead of a mutable status field. Every fact traces to a raw event; duplicates and out-of-order arrival are properties of the fold; the feed and the user's commands write to the same log. It is a

function, not a framework.

- *Provenance in the type system.* A milestone's `actual` slot only accepts an operator-confirmed stamp; a model estimate cannot be assigned to it, and the one date component switches exhaustively on provenance.
- *Exceptions are derived on read, not stored as tickets.* They close when facts arrive, and "done" is derived from logged commands. The first requirement that forces a stored case is assignment.
- *Perimeter inside the gateway*, with a narrower view type for customers: out of perimeter is indistinguishable from not found, and a component cannot forget to filter.
- *Staleness is "an expected update is overdue"*, not hours of silence: a vessel reports nothing for two weeks at sea.
- *Lateness is counted in local days at the destination*, and every event is shown in the local time of its place.

Freshness. The gateway emits hints (which shipments changed) and the UI refetches through the same scoped queries: the shape of server-sent events. Appended events are persisted and broadcast across tabs, so operations and the portal can sit side by side and a notice arrives live.

Tests. About 830 unit and integration tests on a fixed clock, weighted to where a wrong answer misleads: the fold (any order, duplicates, no back-fill), a golden test that derives the state of all 23 shipments from raw operator payloads, perimeter (search cannot leak), each adapter's mapping table, and one Playwright spec for the demo flow.

Prototype	Production
In-memory log, seed parsed at start-up	PostgreSQL: raw events verbatim, canonical events, a projection updated in the same transaction; mapping fixes replay raw events
Four scripted events	One connector per operator (webhook, poller, SFTP) feeding the same adapters
`subscribe` hints, cross-tab broadcast	Transactional outbox and server-sent events filtered by the session's scope
Persona switcher	OIDC sessions; scope compiled to a SQL predicate, row-level security as the second lock
Mock AI adapters	A model gateway (routing, timeouts, caching, logging, budgets) behind the same five ports
Staleness checked on read	An expected-next-update timestamp per shipment and a scheduled sweep

---

4. Assumptions, limitations and next steps

Assumptions. Shipments, plans and committed dates come from the ERP or TMS. One consignment is one container or one set of pallets. Sea shipments are DAP, so the consignee is the importer of record: Estela never proposes "clear customs", it proposes sending our document to the broker. A subsidiary is an account like any other. Every operator exposes some feed, however poor.

Limitations. The estimator is rule-based and says so. There is no holiday calendar. Four scripted events stand in for a feed. Operator messages that cannot be parsed are quarantined but not yet shown to the desk. Documents are metadata. Notices are in English only. Desktop only. State lives in the browser.

Next steps, in order. Real ingestion and the ledger. Delivery appointments and carrier release as milestones: the two things an importer actually plans on. A mobile view for customers. Notices in the customer's language. Assignment, which is what forces stored cases. The learned estimator, once enough shipments per lane have completed to calibrate it.

---

5. How it was built

Specification first, AI coding agents second, checks throughout.

- **Design.** Five independent proposals, each written from one lens (product, AI value, domain and architecture, synthetic data, visual design), then two adversarial reviews: one as a CTO, one as a logistics veteran. The reviews cut the scope sharply and corrected the domain: one ocean lane instead of two, no invented percentages, a clock frozen on a working day, DAP semantics.
- **Specification.** The outcome was a written specification in six parts (architecture, domain, world, application, interface, testing), published in `docs/spec` with the list of where the code departs from it. Every implementation agent started from it, so the context each one worked with was the same and was small.
- **Implementation.** Agents worked in stages, with disjoint file ownership and a written hand-over between stages: inner layers, outer layers, the design system in parallel, then the three screens in parallel against the gateway's view types.

- **Verification.** Nothing was accepted on an agent's word: lint-enforced layer boundaries, a golden test of all 23 shipments written from the specification, mutation spot checks against the test suite, screenshots read before sign-off, and an end-to-end test that caught a real state bug between the two role layouts.
- **Review.** Three independent reviewers then read the core: one for the domain rules, one for the data and the operator adapters, one as the interviewing CTO. They reported 29 findings, most with a failing probe test. Among them: every customs hold was treated as a wrong invoice, a failing estimator turned an at-risk shipment green, a carrier date already past stayed the best one, and this document claimed more than the code enforced. The defects were fixed at their cause, each with a regression test, and the claims were corrected.

That method explains the size of the repository.